現代與後量子密碼學導論

Introduction to Modern and Post-quantum Cryptography

Fall 2026

Part 1: Provable Security

1.2: The Provable Security Mindset

1.3: Provable Security & Computational Cryptography

Zi-Yuan Liu

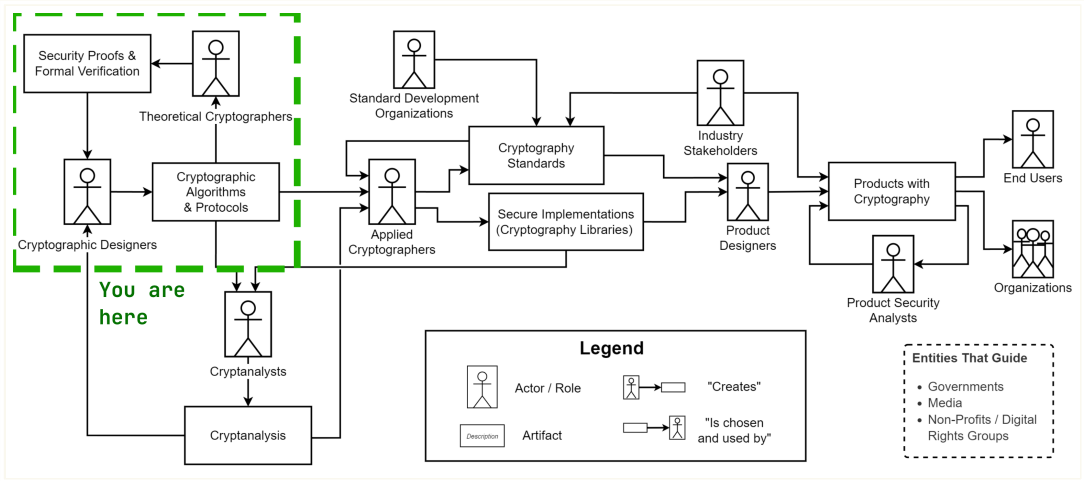
CC BY NC SA

The original slides were prepared by Nadim Kobeissi.

Section 1

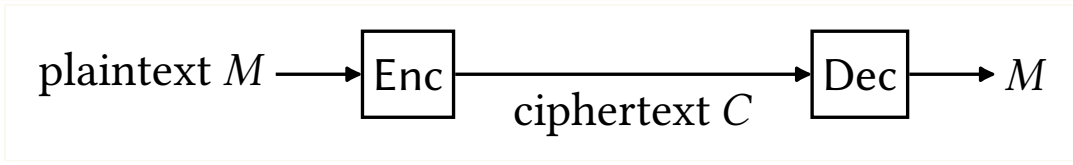
The Provable Security Mindset

How it's made



Fischer et al., The Challenges of Bringing Cryptography from Research Papers to Products: Results from an Interview Study with Experts, USENIX Security 2024

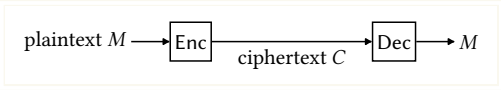
Thinking about secrecy



Source: The Joy of Cryptography

Thinking about secrecy

- Keep the whole design secret?
- **“Advantages”**:
  - Attacker doesn’t know how our cipher (or system, more generally) works.
- **Disadvantages**:
  - Figuring out how the system works might mean a break.
  - Can’t expose cipher to scrutiny.
  - Everyone needs to invent their own cipher.

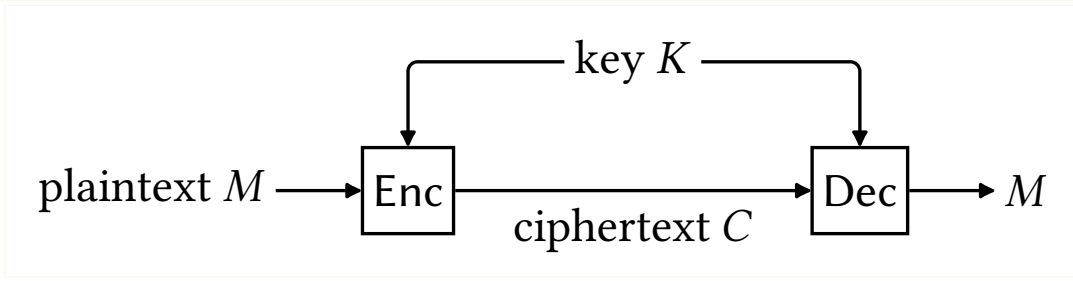


Source: The Joy of Cryptography

Kerckhoffs' (柯克霍夫) principle

- *"A cryptosystem should be secure even if everything about the system, except the key, is public knowledge."* — Auguste Kerckhoffs, 1883
- **Why it matters**:
  - No "security through obscurity"
  - The key is the only secret: the rest can be audited, tested, trusted
  - Encourages open standards and peer review
  - If your system's security depends on nobody knowing how it works, it's not secure.

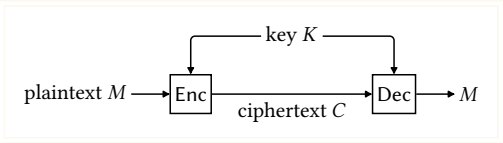
Thinking about secrecy



Concentrate all the need for secrecy in the key!

Thinking about secrecy

- Cipher can be scrutinized and used by anyone.
- Security can be shown to hold so long as the key is secret.
- This is how virtually all cryptography is designed today.



Source: The Joy of Cryptography

One-time pad

First look at a symmetric cipher

$\underline{\textsc{Enc}(K, M)}$:

$C := K \oplus M$

return C

$\underline{\textsc{Dec}(K, C)}$:

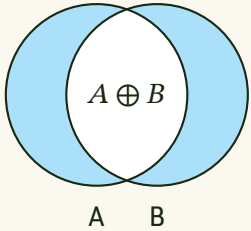
$M := K \oplus C$

return M

XOR (Exclusive OR) operation

A	B	A ⊕ B
0	0	0
0	1	1
1	0	1
1	1	0

Table: Truth table for XOR operation



- XOR returns 1 when inputs differ
- XOR returns 0 when inputs are the same
- Key property: $x \oplus x = 0$ and $x \oplus 0 = x$
- Self-inverse: $(M \oplus K) \oplus K = M$

One-time pad

First look at a symmetric cipher

$$
\begin{array}{cll}
 & 11101111101111100011 & M \\
\oplus & 00011001110000111101 & K \\
\hline
= & 11110110011111011110 & C = \mathsf{Enc}(K, M)
\end{array}
$$

(We're encoding the message and key as bits)

One-time pad

First look at a symmetric cipher

$$
\begin{array}{lll}
 & 11110110011111011110 & C \\
\oplus & 00011001110000111101 & K \\
\hline
= & 11101111101111100011 & M = \mathsf{Dec}(K, C)
\end{array}
$$

(We're encoding the message and key as bits)

Key derivation

Ideal model: uniform sampling

- An attacker should not be able to predict the key K.
- Ideally, K is sampled uniformly at random: $K \xleftarrow{\$} \{0,1\}^n$.
- Every possible n-bit key is equally likely:

$$\Pr[K = k] = 2^{-n} \qquad \text{for every } k \in \{0,1\}^n.$$

Example: a two-bit key

If $n = 2$, then $K \in \{00, 01, 10, 11\}$. Each possible key occurs with probability $\frac{1}{4}$.

Key derivation

In practice: pseudorandom keys

- True randomness is difficult to obtain at scale.
- In practice, keys are generated using a cryptographically secure pseudorandom number generator (CSPRNG):

$$\text{unpredictable seed} \longrightarrow \text{CSPRNG} \longrightarrow K.$$

- In practice, this just means we need the bits to be random, unpredictable, and uniformly distributed.
- **Random:** The bits are not fixed in advance.
- **Unpredictable:** An attacker cannot efficiently guess them.
- **Uniformly distributed:** Every possible key is equally likely.

One-time pad

Correctness proof

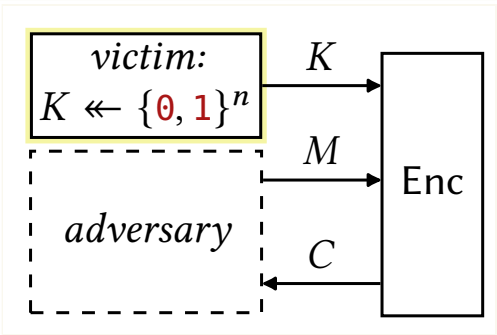
- $\forall (n > 0,\ K \in \{\texttt{0},\texttt{1}\}^n,\ M \in \{\texttt{0},\texttt{1}\}^n),\ \mathsf{Dec}(K, \mathsf{Enc}(K, M)) = M$
- For all positive n: encrypting any n-bit message under any n-bit key, then decrypting under the same key, returns the original message.
- **Proof**:

$$\begin{aligned}
\mathsf{Dec}(K, \mathsf{Enc}(K, M)) &= \mathsf{Dec}(K, K \oplus M) \\
&= K \oplus (K \oplus M) \\
&= (K \oplus K) \oplus M \\
&= \texttt{0}^n \oplus M \\
&= M \qquad \square
\end{aligned}$$

One-time pad

How do we prove security?

- When we prove security, we prove what is or isn't possible for an attacker who calls Attack(M).

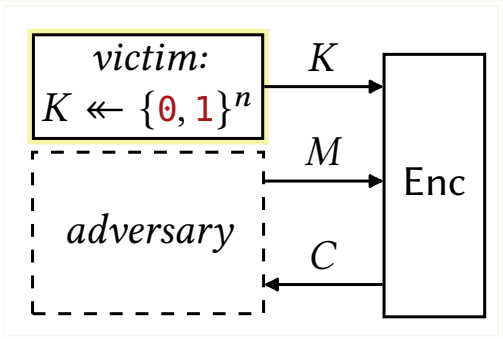


Source: The Joy of Cryptography

One-time pad

How do we prove security?

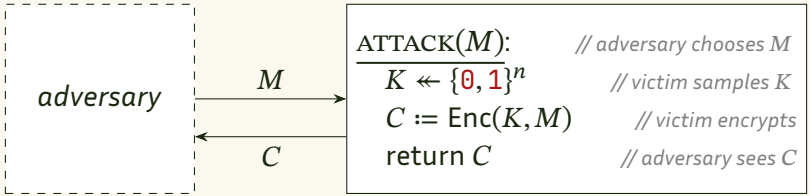
- "Victim" chooses their key.
  - Fresh key for each message (each key used only once)
  - This means the output will differ even if the adversary inputs the same plaintext twice
- Adversary chooses the message and receives the ciphertext.
- We say that **the adversary has access to an encryption oracle**.



Source: The Joy of Cryptography

One-time pad

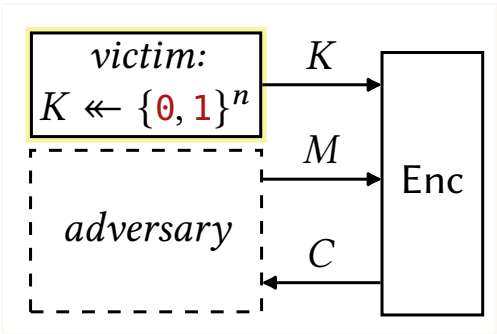
How do we prove security?



One-time pad

How do we prove security?

- Adversary can query oracle an unbounded number of times.
- Two queries with the same M (almost always) return different (C, C'), since the victim samples a fresh K each time.
- K is always chosen correctly (pseudo-random uniform sampling).
- "Randomized oracle"
- Attacker cannot see K.



Source: The Joy of Cryptography

“If I use OTP according to the attack scenario (I sample keys uniformly and use each key to encrypt just one message), then no matter how the plaintexts are chosen, and no matter how the ciphertext is subsequently used, I can enjoy a certain security guarantee.”

— The Joy of Cryptography

One-time pad

How do we prove security?

- **Generally**: a cipher is secure if the adversary can't distinguish the output of calls to Attack from random junk.
- **Formally**: For all positive integers n and all choices of plaintext $M \in \{0,1\}^n$, the output of the following subroutine is uniformly distributed:

$\textsc{Attack}(M)$:

$K \twoheadleftarrow \{0,1\}^n$

$C := K \oplus M$

return C

One-time pad

How do we prove security?

- If the key is random, the output will be uniformly distributed!
- Suppose $M = \texttt{01}$:
  - $K = \texttt{00}$ is chosen with probability $1/4$: $C = K \oplus M = \texttt{00} \oplus \texttt{01} = \texttt{01}$.
  - $K = \texttt{01}$ is chosen with probability $1/4$: $C = K \oplus M = \texttt{01} \oplus \texttt{01} = \texttt{00}$.
  - $K = \texttt{10}$ is chosen with probability $1/4$: $C = K \oplus M = \texttt{10} \oplus \texttt{01} = \texttt{11}$.
  - $K = \texttt{11}$ is chosen with probability $1/4$: $C = K \oplus M = \texttt{11} \oplus \texttt{01} = \texttt{10}$.

$\underline{\textsc{Attack}(M):}$

$K \twoheadleftarrow \{\texttt{0},\texttt{1}\}^n$

$C := K \oplus M$

return C

One-time pad

What's so special about XOR?

- Let's replace $\oplus$ with $\wedge$. What would happen?
- Output no longer uniform!

A	B	A $\wedge$ B
0	0	0
0	1	0
1	0	0
1	1	1

Table: Truth table for AND operation

$\underline{\textsc{Attack}(M):}$

$K \twoheadleftarrow \{0,1\}^n$

$C := K \wedge M$

return C

One-time pad

How do we prove security?

- What if this is true only for $M = \texttt{01}$?
- Fine, let's pick any $M, C \in \{\texttt{0},\texttt{1}\}^n$.
- What is $\Pr[\text{Attack}(M) = C]$?
- $\text{Attack}(M)$ returns C exactly when $C = \text{Enc}(K, M) = K \oplus M$...
- ...which occurs for exactly one key: $K = M \oplus C$.
- Since K is chosen uniformly from $\{\texttt{0},\texttt{1}\}^n$, the probability of choosing that K is $\frac{1}{2^n}$. □

$\textsc{Attack}(M)$:

$K \twoheadleftarrow \{\texttt{0},\texttt{1}\}^n$

$C := K \oplus M$

return C

One-time pad

From the adversary's perspective...

$\textsc{Attack}(M)$:

$K \twoheadleftarrow \{0,1\}^n$

$C := K \oplus M$

return C

$\approx$

(indistinguishable from)

$\textsc{Junk}(M)$:

$C \twoheadleftarrow \{0,1\}^n$

return C

“Real or random?”

One-time pad

What about addition modulo n?

- Let's replace $\oplus$ with addition modulo n. What would happen?
- Still good!
- Can you prove correctness and security?

$\underline{\text{ATTACK}(M)}$:

$K \twoheadleftarrow \mathbb{Z}_n$

$C := (K + M) \bmod n$

return C

Section 2

Breaking One-Time Pads

Two-time pads

What happens when we reuse a key?

- Alice sends two messages with the same key:
- $C_1 = M_1 \oplus K$
- $C_2 = M_2 \oplus K$
- What can Eve learn from C_1 and C_2?
- $C_1 \oplus C_2 = (M_1 \oplus K) \oplus (M_2 \oplus K)$
- $C_1 \oplus C_2 = M_1 \oplus M_2 \oplus (K \oplus K)$
- $C_1 \oplus C_2 = M_1 \oplus M_2$
- The key cancels out completely!

Exploiting $M_1 \oplus M_2$

Why is this dangerous?

- Knowing $M_1 \oplus M_2$ allows statistical attacks:
- **Language patterns**: Natural language has predictable structure
  - Common words: “the”, “and”, “of”, “to”
  - Letter frequencies: E is most common in English (12.7%)
  - Spaces are very frequent (about 18% of characters)
- **Crib dragging**: Try XORing common words at different positions
- **Example**: If we guess “the” at position i in M_1:
  - Extract: $(M_1 \oplus M_2)[i : i + 3] \oplus$ “the”
  - If the result looks like valid text, we may have found part of M_2!

Malleability of one-time pads

Another practical weakness

- OTP provides confidentiality but not integrity
- **Malleability**: Attacker can modify ciphertext predictably
- Given $C = M \oplus K$
- Attacker creates $C' = C \oplus \Delta$
- Decryption: $C' \oplus K = (M \oplus K \oplus \Delta) \oplus K$
- Result: $M' = M \oplus \Delta$
- Attacker controls the change without knowing M or K!

Practical attack

Bit flipping

- **Scenario**: Bank transfer message
- Original: “Transfer $0100.00 to Account 12345”
- Attacker knows position of amount field
- Flips a bit to change ‘1’ (ASCII 49 = `00110001`) to ‘9’ (ASCII 57 = `00111001`)
- XOR difference: `00110001` $\oplus$ `00111001` = `00001000`
- Apply Δ = `00001000` at the correct position in the ciphertext
- Result: “Transfer $0900.00 to Account 12345”
- No need to break encryption or know the key!

Section 3

Limitations of Security Proofs

Limitations of security proofs

Part 1

- Rigor and the real world famously don't mix.
- Security proofs are good for rigor but say very little about real-world concerns:
  - How can Alice and Bob obtain a secret key that only they know?
  - How can they keep K secret?
  - How can a computer sample from the uniform distribution?
  - How can Alice ensure that C is sent reliably to Bob?

Limitations of security proofs

Part 2

- More questions proofs don't address:
  - How can Alice hide the fact that she is talking to Bob (rather than hide only the content)?
  - How can Alice be sure that she is communicating with Bob, not an impostor?
  - How can we incentivize Alice and Bob to use encryption?
  - Should the government be allowed to obtain a warrant to read encrypted communications?
- Security proofs are about specific properties within specific models.
- Real-world security depends on many factors beyond what our models capture.
- Having a security proof is necessary but not sufficient for real-world security.

The value of security proofs

- Despite limitations, security proofs provide important benefits:
  - **Precise guarantees**: Clearly define what security properties are achieved.
  - **Confidence**: A valid proof rules out all attacks within the model.
  - **Foundation for composition**: Proven components can be securely combined.
  - **Precise terminology**: Forces us to clearly define our terms and assumptions.
- Security proofs help identify the *boundaries* of security:
  - What assumptions are necessary?
  - What threats are addressed vs. unaddressed?
  - What conditions must hold for security to be maintained?

The provable security mindset

- Building systems with provable security in mind:
  - Start with clear security goals and adversary model.
  - Design systems whose security can be formally analyzed.
  - Identify and document necessary assumptions.
  - Distinguish between proven properties and conjectures.
- Good practical security requires both:
  - Rigorous proofs for core mechanisms.
  - Practical engineering to address real-world constraints.

OTP: security assumptions & constraints

Part 1

- Our security proofs rely on specific assumptions about the adversary:
  - **Key reuse**: Keys are never intentionally reused (though may repeat by chance).
  - **Observation only**: Adversary passively observes the ciphertext but doesn't tamper with it.
  - **Message independence**: Choice of message M is independent of key K.
  - **Key secrecy**: Adversary learns nothing about the key.
  - **No sampling influence**: Adversary cannot influence how the key is sampled.
- These constraints are *necessary* for the security proofs to hold!

OTP: security assumptions & constraints

Part 2

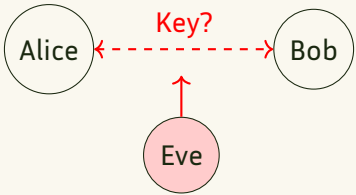
- Side-channel attacks violate our model:
  - We assume the adversary cannot coerce the victim into running a different algorithm.
  - Cannot observe execution details:
    - CPU timing information (clock cycles).
    - Memory access patterns.
    - Cache hits/misses.
    - Power consumption during encryption.
- Real-world security requires considering these additional attack vectors.
- Our security proofs address a *specific threat model* that may not capture all real-world threats.

Key distribution problem

The elephant in the room

- OTP requires a key as long as the message
- How do Alice and Bob share this key?
- **The paradox**:
  - If you can securely send the key...
  - ...why not just send the message?
- **Historical solutions**:
  - Diplomatic pouches
  - Trusted couriers
  - Pre-shared codebooks
- **Modern issue**: Impractical at internet scale

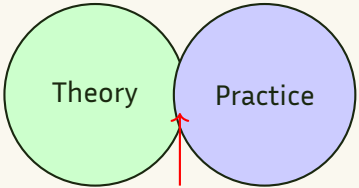
Problem: How to share the key securely?



If the channel is secure for the key, why not use it for the message?

From theory to practice

- **Theory provides foundation**:
  - Precise security definitions
  - Mathematical guarantees
  - Clear assumptions
- **Practice reveals challenges**:
  - Implementation vulnerabilities
  - Usability constraints
  - Side-channel attacks
  - Human factors



Sweet spot:
Provably secure &
practically usable

Cryptographic primitives

- Specific algorithms like OTP are important in cryptography, but OTP is just one instance of an **encryption scheme**.
- In cryptography, useful abstractions like “encryption scheme” are called **primitives**.
- Three things are important when defining a cryptographic primitive:
  1. **Syntax**: The basic raw interface - algorithms, inputs/outputs, and their types.
  2. **Correctness**: Basic functionality without adversaries - e.g., “decryption should be the inverse of encryption”.
  3. **Security**: Guarantees that hold in specific attack scenarios with adversaries.

Cryptographic primitives

Σ: a symmetric-key encryption scheme

- $\Sigma.\textbf{KeyGen}() = K$
  - Input: none
  - Output: key $K \in \Sigma.\mathcal{K}$ (the "key space").
- $\Sigma.\textbf{Enc}(K, M) = C$
  - Input: key $K \in \Sigma.\mathcal{K}$, plaintext $M \in \Sigma.\mathcal{M}$ (the "message space").
  - Output: ciphertext $C \in \Sigma.\mathcal{C}$.
- $\Sigma.\textbf{Dec}(K, C) = M$
  - Input: key $K \in \Sigma.\mathcal{K}$, ciphertext $C \in \Sigma.\mathcal{C}$ (the "ciphertext space").
  - Output: plaintext $M \in \Sigma.\mathcal{M}$.

Correctness of an SKE

Correctness for SKE

An SKE scheme Σ is correct if encryption and decryption are inverses, in the following sense:

$$\Pr\left[\Sigma.\mathsf{Dec}(K, \Sigma.\mathsf{Enc}(K, M)) = M\right] = 1$$

for all $M \in \Sigma.\mathcal{M}$ and $K \in \Sigma.\mathcal{K}$.

- The definition involves a probability because $\Sigma.\mathsf{Enc}$ may be a randomized algorithm.
- This means that decryption should **always** recover the original message.
- Even if encryption is randomized, decryption must always map each key-ciphertext pair back to the original message.

One-time secrecy of an SKE

One-time Secrecy for SKE

An SKE scheme Σ has one-time secrecy if the following libraries are interchangeable:

$\mathcal{L}^{\Sigma}_{\text{ots-real}}$

$\textsc{ots.enc}(M)$:
$K \twoheadleftarrow \Sigma.\mathcal{K}$
$C := \Sigma.\text{Enc}(K, M)$
return C

$\equiv$

$\mathcal{L}^{\Sigma}_{\text{ots-rand}}$

$\textsc{ots.enc}(M)$:
$C \twoheadleftarrow \Sigma.\mathcal{C}$
return C

An encryption scheme has one-time secrecy if its ciphertexts are uniformly distributed when keys are sampled uniformly, kept secret, and used for only one encryption, no matter how the plaintexts are chosen.

Reminder: AND ($\wedge$)

- Let's replace $\oplus$ with $\wedge$. What would happen?
- Output no longer uniform!

A	B	A $\wedge$ B
0	0	0
0	1	0
1	0	0
1	1	1

Table: Truth table for the AND operation

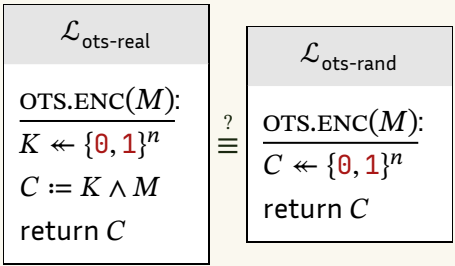
$\underline{\text{ATTACK}(M):}$

$K \twoheadleftarrow \{0,1\}^n$

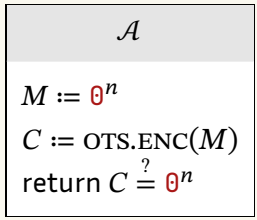
$C := K \wedge M$

return C

Creating a distinguisher program



- Can you write a distinguisher program showing that these libraries are **not** interchangeable?



- $\Pr\left[\mathcal{A} \diamond \mathcal{L}_{\text{ots-real}} \Rightarrow \texttt{true}\right] = 1$
- $\Pr\left[\mathcal{A} \diamond \mathcal{L}_{\text{ots-rand}} \Rightarrow \texttt{true}\right] = \frac{1}{2^n}$

Section 4

Computational Cryptography

Computational notions in cryptography

- Security definitions in theoretical cryptography can be too strict:
  - A scheme counts as “broken” even if the only attacks require a trillion years of computation.
  - Or even if attacks succeed with probability lower than that of winning the lottery 100 times in a row.
- Modern provable security takes a more practical approach:
  - We dismiss attacks with “ridiculously” high computational cost.
  - We dismiss attacks with “ridiculously” tiny success probability.
- Instead of “no attack can succeed, not even in principle”,
- We prove: “every attack has either ridiculously high cost or ridiculously small success probability”.

The concrete approach to provable security

- In the concrete approach to provable security, we aim to be as quantitative as possible about security claims.
- We rarely say definitively that a cryptographic algorithm “is secure” (though we did for OTP).
- Instead, we use statements like:
  - *“Any attack that expends at most 2^{80} effort can succeed with probability no better than 2^{-64}.”*
- It is up to the user to judge whether this quantitative level of security is acceptable based on their use case.
- This gives users a concrete basis for security decisions.

Monetary cost of huge computations

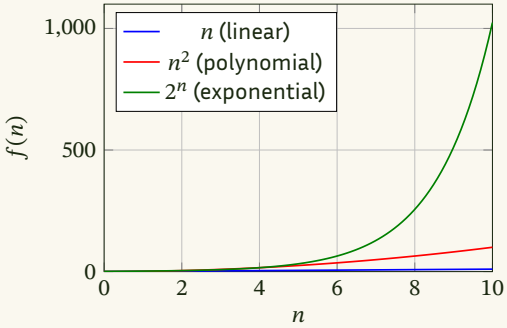
- One way to think about huge computations is their monetary cost.
- The table below shows roughly how much a computation involving 2^n CPU cycles would cost on the cheapest available Amazon EC2 cloud computing service:

Clock cycles	Approx cost (USD)	Point of reference
2^{50}	$3.50	cup of coffee
2^{55}	$100	dinner at a high-end restaurant
2^{65}	$130,000	down payment on apartment
2^{75}	$130 million	budget of one of the Harry Potter movies
2^{85}	$140 billion	GDP of Hungary
2^{92}	$20 trillion	GDP of the United States
2^{99}	$2 quadrillion	all of human economic activity since 300,000 BCE
2^{128}	a lot!	a billion human civilizations' worth of effort

Exponential vs polynomial growth

Quick side-note

- Important insight: $2^{n+1} = 2 \times 2^n$
  - $2^{51} = 2 \times 2^{50}$
  - $2^{128} = 2^{64} \times 2^{64}$
- Each additional bit doubles the computational cost!
- Compare growth rates:
  - Linear: $f(n) = n$
  - Polynomial: $f(n) = n^2$
  - Exponential: $f(n) = 2^n$



Note: Exponential growth quickly surpasses all polynomial functions!

Understanding tiny probabilities

- Let's put extremely small probabilities in perspective:
  - In 2009, Patricia Demauro rolled dice 154 consecutive times in craps without getting a 7.
  - Probability of this event: $(30/36)^{154} \approx 2^{-40.5}$
  - Often cited as one of the most improbable documented events in gambling history.
- Other extremely unlikely gambling events:
  - In 1943, a roulette wheel reportedly landed on red 32 consecutive times.
  - Probability: $(18/38)^{32} \approx 2^{-34.5}$
  - Winning the American Powerball lottery: $2^{-28.1}$
  - Winning Powerball two consecutive weeks: $2^{-56.2}$
- These examples help us intuitively grasp what probabilities like 2^{-40} or 2^{-50} actually mean.

Why 2^{-x} for probabilities?

Quick side-note

- We've been using notation like 2^{-40}, 2^{-64}, 2^{-128} for probabilities. Why the negative exponent?
- Recall that $2^{-x} = \frac{1}{2^x}$
  - $2^{-40} = \frac{1}{2^{40}} = \frac{1}{1{,}099{,}511{,}627{,}776}$
  - This is approximately 1 in a trillion.
- Probabilities must be between 0 and 1:
  - 2^{40} would be a huge number (over a trillion) - not a valid probability!
  - 2^{-40} is a tiny fraction - exactly what we want for unlikely events

Computational scale of Bitcoin mining

- Let's consider cryptocurrencies based on proof-of-work, like Bitcoin.
- These systems incentivize users to perform truly obscene amounts of computation.
- In Bitcoin's proof-of-work mechanism:
  - Users race to perform SHA-256 hash computations as fast as possible.
  - The collective Bitcoin network has performed approximately 2^{95} SHA-256 hashes in total.
  - In the last 12 months alone: approximately $2^{93.6}$ hashes.
  - Current market cap for Bitcoin: approximately 1.7 trillion USD.
- This shows the enormous scale of computation that economic incentives can support.

The asymptotic approach to provable security

- The concrete approach gives practical guidance (e.g., key sizes) but requires managing tedious quantitative details.
- The asymptotic approach[a]:
  - Makes qualitative, all-or-nothing statements
  - Considers behavior as key size approaches infinity
  - Hides tedious quantitative details
- Similar to asymptotic analysis using big-O notation.
- Example: Instead of calculating that an operation takes exactly $16n^2 + 24n + 74$ steps, we can simply say it takes $\mathcal{O}(n^2)$ steps.

[a] Asymptotic means "approaching a limit". In computer science, we study how algorithms behave as input size approaches infinity, focusing on the general trend rather than exact values.

AES: An example of asymptotic security analysis

- The Advanced Encryption Standard (AES) is a common symmetric-key encryption algorithm.
- It comes in different variants: AES-128, AES-192, AES-256.
- In asymptotic analysis, we'd say:
  - AES is secure against all polynomial-time adversaries.
  - Any successful attack must take exponential time in the key length.
- This hides the concrete details about specific computational costs.

- Concrete security for AES-128:
  - Best known attack: $\approx 2^{126.1}$ operations
- For AES-256:
  - Best known attack: $\approx 2^{254.4}$ operations
  - Far beyond the capability of any conceivable computation
- Asymptotic approach: Both are "computationally secure"

Why does a 128-bit key mean 2^{128} possible keys?

- Each bit can be either 0 or 1
- For AES-128:
  - Key has 128 bits
  - Each bit: 2 choices
  - Total combinations: $\underbrace{2 \times 2 \times 2 \times \ldots \times 2}_{128 \text{ times}} = 2^{128}$
- That's approximately 3.4×10^{38} different keys!

Polynomial running time

- In the asymptotic approach, we focus only on adversaries that run in polynomial time.
- The security parameter λ determines the level of security:
  - Usually the length of secret keys in bits
  - All algorithms have access to λ as a global variable
  - Example: AES-128 uses $\lambda = 128$, AES-256 uses $\lambda = 256$ ("256-bit security").

Polynomial Running Time

An algorithm runs in polynomial time if there is a polynomial p such that the algorithm takes at most $p(n)$ steps on inputs of length n.

Negligible functions

Negligible Functions

A function f is negligible if it approaches zero faster than $1/p(\lambda)$, for every polynomial p. Formally:

- For every polynomial p, there exists λ_0 such that $f(\lambda) < 1/p(\lambda)$ for all $\lambda > \lambda_0$.
- For every polynomial p, we have $p(\lambda) \in \mathcal{O}(1/f(\lambda))$.

- **You can probably ignore the definition and just think about this intuitively:**
- In cryptography, we want attack success probabilities to be negligible in the security parameter λ.
- Negligible probability: essentially zero for practical purposes when λ is large enough.

Indistinguishability in computational security

Indistinguishability

Let $\mathcal{L}_1$ and $\mathcal{L}_2$ be two libraries with the same interface. The **advantage** of a calling program $\mathcal{A}$ in distinguishing $\mathcal{L}_1$ and $\mathcal{L}_2$ is:

$$\left|\Pr\left[\mathcal{A} \diamond \mathcal{L}_1 \Rightarrow \texttt{true}\right] - \Pr\left[\mathcal{A} \diamond \mathcal{L}_2 \Rightarrow \texttt{true}\right]\right|$$

We write $\mathcal{L}_1 \approx \mathcal{L}_2$ if every polynomial-time calling program has only negligible advantage in distinguishing them.

- Previously, libraries were interchangeable when probabilities were **identical**.
- Now, libraries are indistinguishable when probabilities are **negligibly close**.
- Again: this is intuitive, so no need to stress about the formal definition.

現代與後量子密碼學導論

Introduction to Modern and Post-quantum Cryptography

Fall 2026

Part 1: Provable Security

1.2: The Provable Security Mindset

1.3: Provable Security & Computational Cryptography

Zi-Yuan Liu

CC BY NC SA

The original slides were prepared by Nadim Kobeissi.